

Návrh a implementace databázových systémů

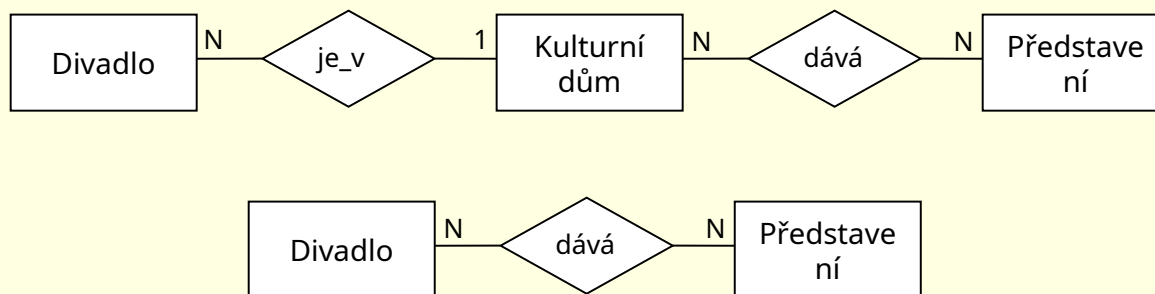
Konceptuální modelování (2)

Integrace pohledů

- několik týmů vytvořilo lokální ER modely
- pro návrh globálního schematu je nutné tato dílčí schemata spojit v jedno
- problémy
 - některé části se ve schemech opakují
 - je třeba unifikovat reprezentaci překrývajících se částí
 - v překrývajících se částech se budou nejspíš vyskytovat konflikty
 - strukturální
 - ve jménech

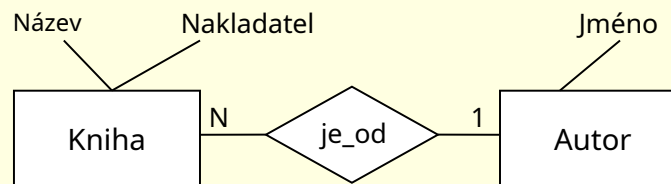
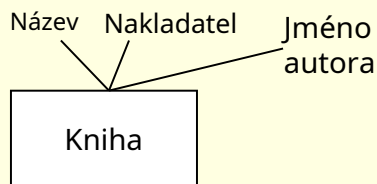
Integrace pohledů

- strukturální konflikt – příklad
 - odlišná úroveň abstrakce



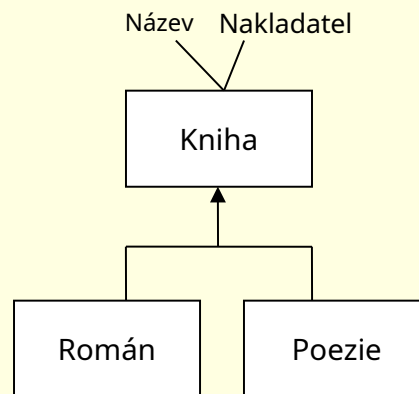
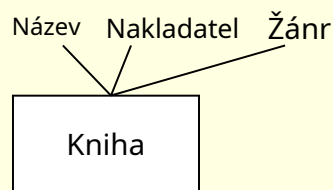
Integrace pohledů

- strukturální konflikt – příklad
 - sémantický relativismus, záměna atributu za vztah



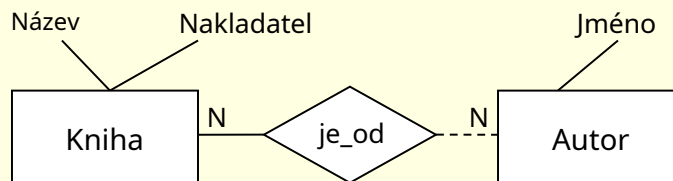
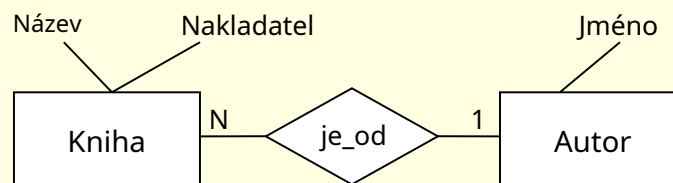
Integrace pohledů

- strukturální konflikt – příklad
 - sémantický relativismus, hierarchie



Integrace pohledů

- strukturální konflikt – příklad
 - nekompatibilní specifikace vztahů
 - rozdílné kardinality
 - rozdílné parciality



Integrace pohledů

■ strukturální konflikt

■ řešení

- obvykle se dává přednost obecnější variantě
- pokud se dva analytici koukají na tentýž problém různě, značí to určitou nejednoznačnost zadání, vybereme to řešení, které je bližší skutečné realitě

■ konflikt jmen

■ synonyma

- dva analytici pojmenují stejnou věc různě
 - dá se odhalit celkem snadno

■ homonyma

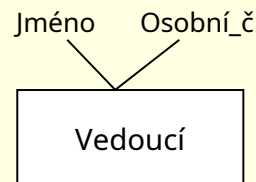
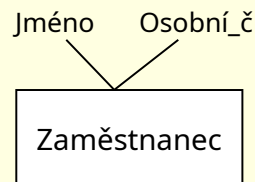
- dva analytici pojmenují různé věci stejně
 - to je horší, na to se přichází obtížněji

Integrace pohledů

- metoda
 - pohledy integrujeme obvykle po dvojicích a tvoříme větší celek
 - nebo k celku přidáváme další a další dílčí schemata
 - záleží na použité strategii návrhu
 - u smíšené strategie se začíná skeletálním schematem
- při integraci nestačí ER diagram, je nutný verbální popis všech použitých pojmů!

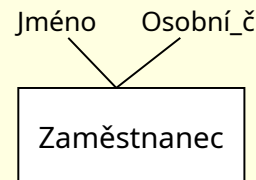
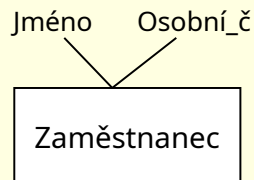
Integrace pohledů

■ konflikt jmen entit – příklad



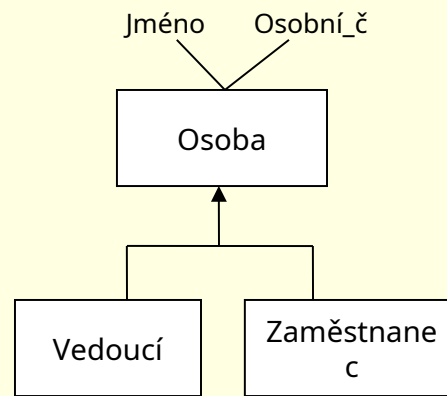
■ nejjednodušší řešení

- konflikt je sice vyřešen, avšak je ignorován zřejmý rozdíl v úrovni abstrakce: vedoucí je patrně podtypem zaměstnance



Integrace pohledů

- lepší řešení



Modelování funkcí

- nedílnou součástí analýzy a návrhu IS je funkční analýza
 - modelování činností týkajících se dané aplikace
- funkční analýza
 - jaké informace vstupují do funkcí
 - jak informace putují mezi funkcemi
 - jaké informace vystupují z funkcí
 - první krok ve specifikaci aplikačních programů, jež budou nad databází operovat
 - komplement k datové analýze
 - umožní ji verifikovat a ověřit úplnost datového modelu

Modelování funkcí

- způsoby modelování funkcí
 - hierarchie funkcí
 - události – spouštěče funkcí
 - DFD – data flow diagramy, diagramy toků dat
 - stavové diagramy
 - Petriho sítě
- různé úrovně abstrakce
 - (pracovní) funkce v organizaci
 - proces v konceptuálním modelu
 - procedura, program v programové úrovni

Co to je funkce

- funkce je něco, co organizace dělá
 - založení nového abonenta v evidenci vstupenek
 - odstranění studenta ze seznamu aktivních studentů
- funkce má mít výstižné jméno a popis
 - identifikuj, změň, odstraň, odešli, spočítej,...
 - např. „založ nový požadavek na rezervaci“
- v popisu funkce specifikujeme CO chceme udělat, nikoli JAK
 - nezmiňují se organizační struktury (oddělení...) či role uživatelů (ředitel...)
 - vyloučit informace typu KDE, KDO, KDY, JAK

Funkční analýza

- funkční analýza
 - objevování funkcí
 - viz metody nalézání faktů
 - postup shora dolů nebo zdola nahoru
 - obojí lze kombinovat a vzájemně doplňovat a testovat
 - schema funkcí má být
 - přesné a srozumitelné
 - úplné
 - stručné ve výrazivu
 - vyvážené

Funkční analýza

- vztah funkcí ke konceptuálnímu schematu
 - během návrhu testovat zda
 - existují funkce, které zmiňují dané entity
 - existují funkce, které vytvářejí, mění či odstraňují entity, vztahy, hodnoty atributů v koncept. schematu
- specifikace funkce
 - popis její logiky, tj. jak se provádí
 - popis algoritmu
 - časově náročné
 - provádí se především u klíčových a složitých funkcí
 - měl by být natolik exaktní, aby umožnil jednoznačnou implementaci

DFD

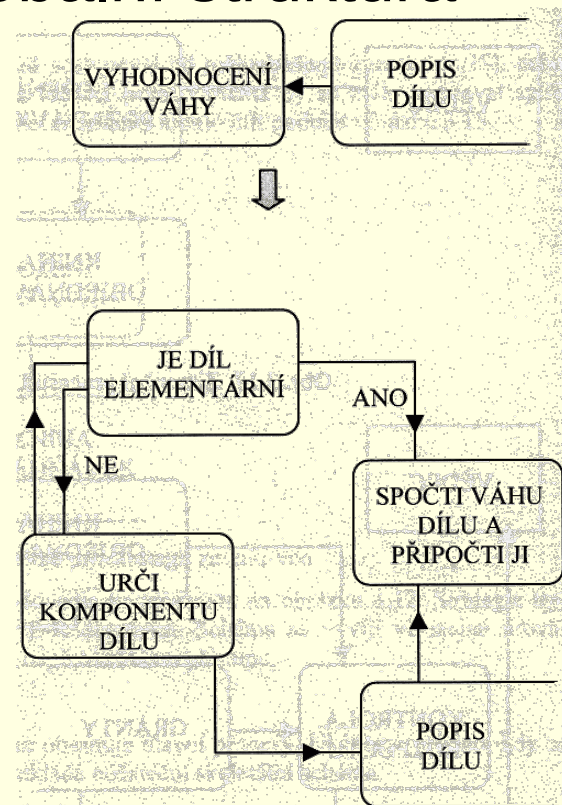
- model toků dat
 - místo funkcí se obvykle hovoří o procesech
- konstrukty
 - proces (process)
 - reprezentuje aktivitu v IS
 - transformuje informace
 - předává informaci jiným procesům
 - může pokrývat několik pracovních funkcí
 - tok dat (data flow)
 - výměna informace mezi procesy
 - sklad dat (data store)
 - místo, kde jsou uložena data – šanon, databáze...

DFD

- rozhraní (interface), někdy se nazývá terminátor
 - např. externí uživatel
 - při dekomponovaném schematu může reprezentovat část IS popsanou jinde
- diagram toků dat
 - funkční popis systému, nikoli popis implementace
 - orientovaný graf
 - uzly: procesy, rozhraní, sklady
 - hrany: toky dat
 - je dobré všechno nějak pojmenovat, i hlavní toky dat
 - někdy lze procesy vnořovat do sebe
 - někdy se (vcelku rozumně) požaduje, aby mezi skladem a rozhraním byl vždy aspoň jeden proces

DFD

- obecná doporučení
 - schema má ukazovat CO, nikoli JAK
 - nezjemňovat, přechází-li v procedurální popis
 - nezjemňovat, ztrácí-li se globální struktura

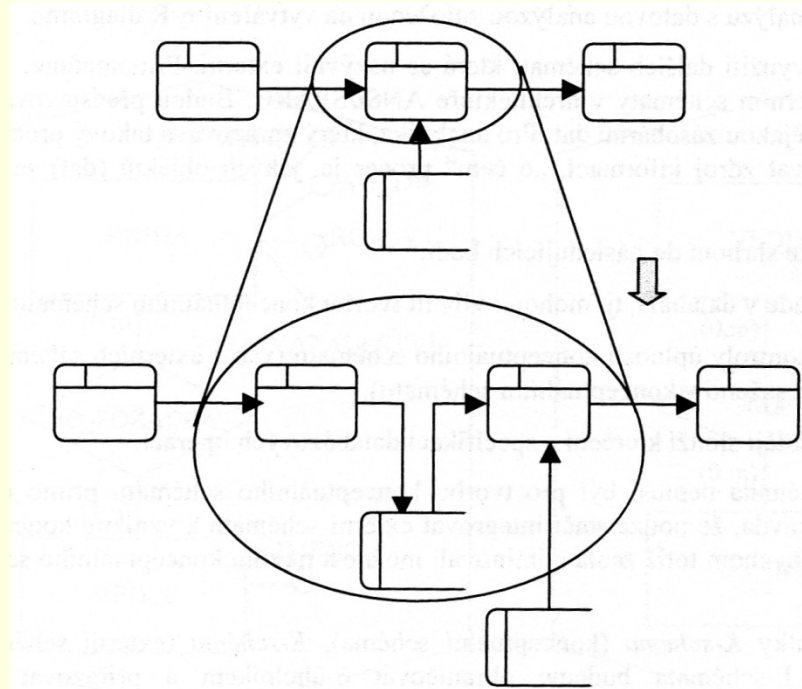


DFD

- posouzení kvality
 - funkční nezávislost
 - každý proces je autonomní, může být analyzován a měněn nezávisle na zbytku systému
 - úplnost
 - DFD pokrývá všechny rysy aplikace
 - korektnost
 - zachování ekvivalence hranic po zjemnění
 - čitelnost
 - minimalita
 - procesy a sklady se nepřekrývají

DFD

- ekvivalence hranic



DFD

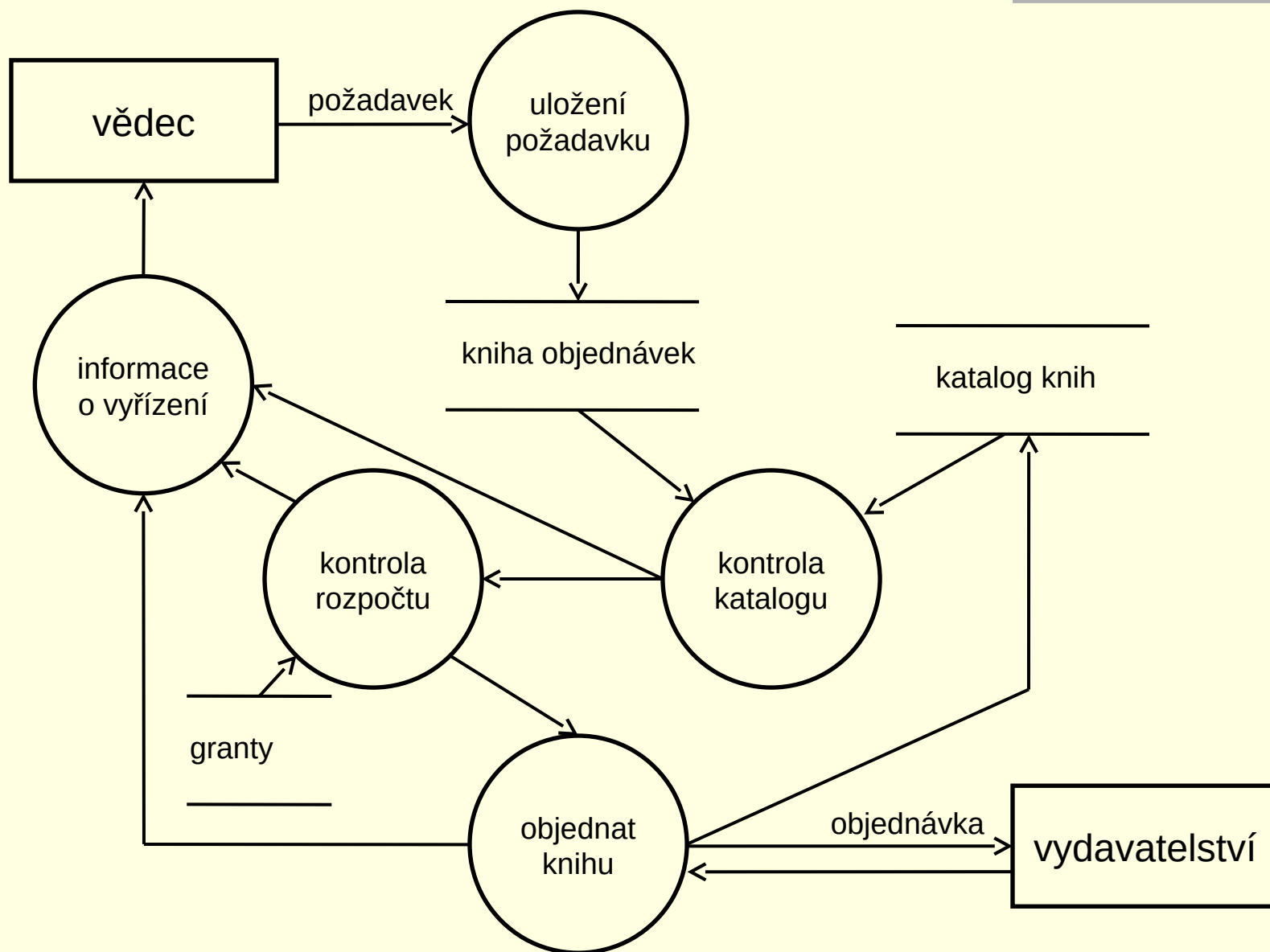
- integrace datové a funkční analýzy
 - vzájemné ovlivňování
 - například zjemnění schematu funkcí vyvolá nutnost zjemnit i ER model
 - např. rozdělíme proces na dva, čímž je indukována nutnost zavést do ER modelu ISA hierarchii
 - kontrola vzájemné úplnosti
 - každý pojem, zmiňovaný v tocích dat a zásobárnách, se vyskytuje v ER schematu
 - každý datový prvek ER modelu musí mít k sobě proces, který jej vytváří, mění, odstraňuje

DFD

■ příklad

- Máme systém vědecké knihovny, vědec si tu a tam přes knihovnu chce objednat knihu. Vědec tedy zašle požadavek do knihovny, tam se požadavek zaknihuje a pak se postupně zpracovává. Pokud je kniha v knihovně, rovnou se mu půjčí. Pokud se musí koupit, zkoumá se, zda ji lze financovat z grantu.

DFD – příklad

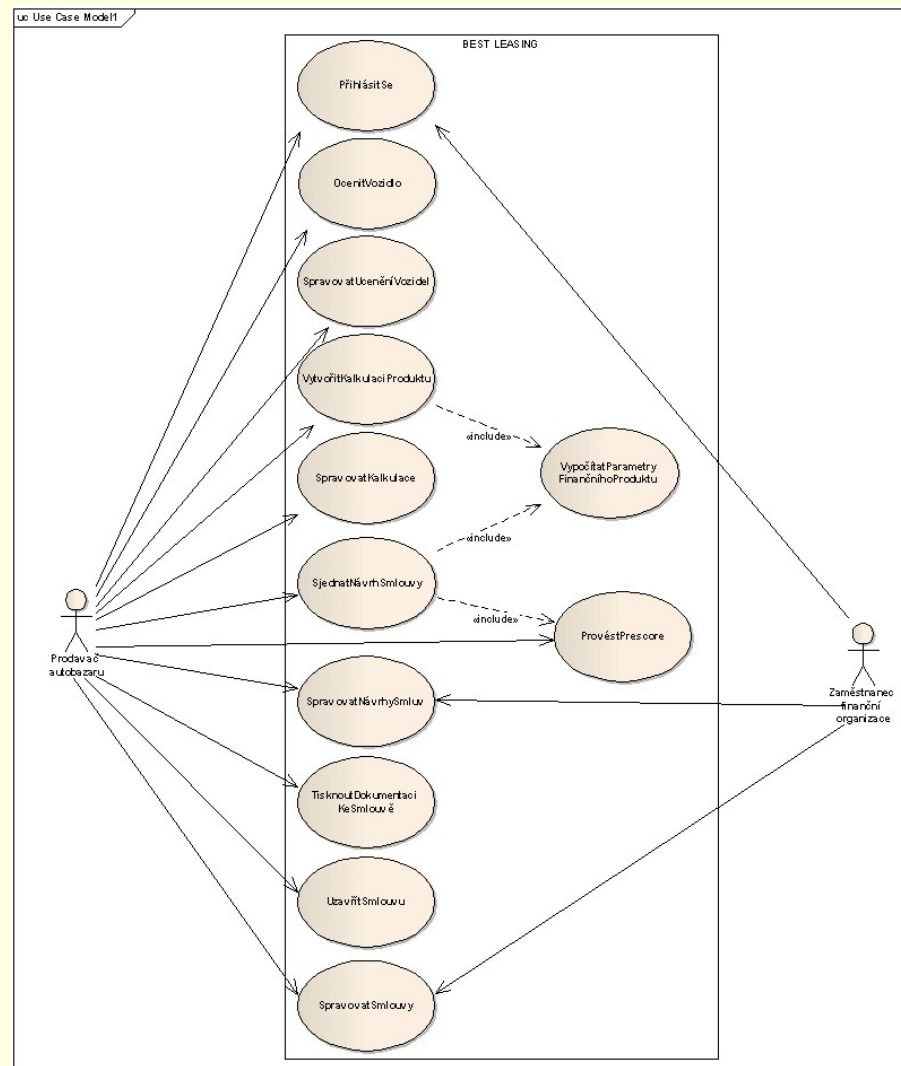


UML

- v modelovacím jazyce UML existují různé typy diagramů pro modelování funkcionality systému
 - diagram případů užití (use case diagram)
 - diagram aktivit
 - stavový diagram
 - sekvenční diagram
 - ...
- vyjdeme-li z jednotlivých uživatelských pohledů, pro každou roli můžeme navrhnout use case diagram
 - jednotlivé případy užití se potom popisují např. diagramem aktivit či sekvenčním diagramem

UML

- use case diagram
 - seznam aktivit, jež daný aktér může provádět
 - obvykle vnímán jako součást *objektového návrhu*, ale lze ho použít pro popis v jakémkoli paradigmatu



UML

■ use case diagram

- aktivity (transakce) musí být dostatečně obecné a zároveň přesně vymezené a konkrétní
 - ani *uživatel* - *stisk tlačítka Submit*,
 - ani *administrátor* - *správa uživatelů systému*
- neformálně vyhlížející diagram svádí k nepřesnostem
 - je třeba důsledně a systematicky pojmenovávat a popisovat – dodržovat nějaké stálé konvence
 - nedodržování jednotných pravidel
 - vede k chybám,
 - snižuje možnost znovupoužití,
 - zpomaluje vývoj systému,
 - ztěžuje testování

UML

■ use case diagram

■ rizika – na co si dát pozor

- je-li více diagramů, obvykle se částečně významově překrývají – konflikty, které je nutno identifikovat a řešit
 - pokud se překryvy neidentifikují, vede to k znásobení práce a nejednoznačností – *ten samý objekt* je v různých use casech implementován *různě*
- use case popisuje systém z hlediska jednoho uživatele, jeho zobrazení na strukturu systému však **není 1:1**
 - jeden objekt je obvykle využit v mnoha use casech
 - jeden use case obvykle používá mnoho objektů
- => jednotlivé diagramy užití jsou tedy pouze lokální modely, za nimiž je potřeba vidět (a vytvořit) jeden globální celek
 - analogie dříve vyložené **integrace pohledů**